

LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 2: Neural Networks - The Foundation
Lecture	Lecture 2.4: Understanding Model Parameters

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Define model parameters and explain what they consist of in a neural network.
- Calculate a simple parameter count for a small network and extrapolate to understand large model scales.
- Explain the relationship between parameter count and model capacity, performance, and resource requirements.
- Describe the trade-offs between model size (more parameters) and practical deployment constraints.
- Articulate the concept of emergent abilities and why large models exhibit qualitatively different capabilities.

Introduction

When the AI field discusses GPT-3's 175 billion parameters or GPT-4's rumoured trillion-plus parameters, these numbers are not marketing hyperbole — they describe a specific, measurable property of the model that directly determines its capabilities, its resource requirements, and its limitations. Understanding what parameters are, how they scale, and why parameter count matters is essential for anyone making decisions about AI adoption, procurement, or deployment.

Parameters are, at their core, the learned numerical contents of a neural network — the weights and biases that are adjusted during training. But the quantity of these numbers has non-trivial consequences. More parameters mean more representational capacity — the ability to capture more complex patterns in data. But they also mean more memory, more compute per inference, more data required for effective training, and higher operational costs. The parameter count is therefore a key engineering and business trade-off, not merely a technical detail.

Perhaps most interestingly, parameter count is associated with the emergence of qualitatively new capabilities at certain scale thresholds. Small language models can complete simple patterns. Medium models can write coherent paragraphs. Large models can reason, translate between languages, write code, and perform multi-step problem-solving — capabilities that appear abruptly at sufficient scale rather than gradually. This emergence phenomenon is what drove the intense investment in very large models and what underpins the current wave of generative AI capability.

Core Concepts Explained

What Model Parameters Are

Definition: Model parameters are the learnable numerical values in a neural network — specifically all the weights (controlling connection strengths between neurons) and biases (offset values for each neuron). They are adjusted during training and collectively encode everything the model has learned.

Business relevance: When a model is described as having X billion parameters, it means it has X billion individual numbers — each playing a specific role in the model's computations. These parameters are what distinguish a trained model from an untrained one; they represent the accumulated result of the training process.

Practitioner relevance: Understanding parameters helps professionals interpret model documentation: parameter counts directly indicate memory requirements, inference compute, and training data needs. A 7-billion-parameter model and a 70-billion-parameter model have very different deployment profiles.

Real-world example: A simple neural network with 10 input neurons, one hidden layer of 5 neurons, and 3 output neurons has 73 parameters: 10 times 5 = 50 weights from input to hidden, 5 biases for hidden neurons, 5 times 3 = 15 weights from hidden to output, and 3 biases for output neurons. GPT-3 has 175 billion such parameters across its 96 transformer layers.

Parameter Count and Model Capacity

Definition: Model capacity refers to the range of functions a model can potentially represent. More parameters enable the model to represent more complex patterns, store more information, and perform better on tasks requiring nuanced understanding. Capacity is necessary but not sufficient for good performance.

Business relevance: Parameter count is a proxy for what the model can potentially learn. However, capacity must be matched to training data: a very large model trained on insufficient data will overfit. The right model size for a task depends on the complexity of the task and the volume of available training data.

Practitioner relevance: For professionals commissioning AI systems, larger is not always better. A smaller, well-trained model often outperforms a much larger model when the task is well-defined and sufficient domain-specific training data is available. The goal is matching capacity to complexity, not maximising parameter count.

Real-world example: A model with 7 billion parameters trained specifically on medical literature often outperforms a general-purpose 70-billion-parameter model on medical question answering — because the smaller model's capacity is fully utilised by the domain-specific training, while the larger model's general training is less efficient for the specific task.

Risks or limitations: Larger models have higher inference latency, higher memory requirements, and higher per-query costs. Deploying a 175-billion-parameter model to answer simple customer service queries when a 7-billion-parameter fine-tuned model would suffice wastes resources and increases operational costs.

Emergent Abilities in Large Models

Definition: Emergent abilities are capabilities that appear in large language models above certain parameter count thresholds but are absent or minimal in smaller models. Examples include multi-step reasoning, in-context learning, code generation, and cross-lingual transfer. These abilities are not explicitly trained — they emerge from scale.

Business relevance: Emergent abilities partially explain why investment in very large models (GPT-3, GPT-4, Claude, Gemini) continued even when smaller models seemed adequate for many tasks. At sufficient scale, models acquire capabilities that cannot be achieved by simply making smaller models better at specific tasks.

Practitioner relevance: Understanding emergence helps set expectations: smaller models can be extremely capable within specific domains but may lack the cross-domain reasoning and generative flexibility of very large models. The right model depends on whether the application needs these emergent capabilities or can be served by task-specific capability.

Real-world example: GPT-2 (1.5 billion parameters) could generate coherent paragraphs but struggled with multi-step arithmetic or consistent logical reasoning over long passages. GPT-3 (175 billion parameters) exhibited dramatic improvements in these areas — capabilities that seem to have emerged from scale rather than from any specific architectural change.

Resource Requirements and Deployment Trade-offs

Definition: Parameter count directly determines the memory required to load a model (roughly 2 bytes per parameter in half-precision), the compute required per inference (scales with parameter count and context length), and the training compute and data required to achieve good performance.

Business relevance: Deployment economics are dominated by inference costs at scale. A 175-billion-parameter model requires hundreds of gigabytes of GPU memory and multiple high-end GPUs for a single instance, costing tens of dollars per hour. A 7-billion-parameter model can run on a single consumer GPU or via cheap cloud APIs. For most business applications, the smaller model at lower cost is more economically rational.

Practitioner relevance: When evaluating AI product proposals, asking for the model's parameter count and intended inference infrastructure allows you to validate deployment feasibility and cost estimates. A proposal to deploy a 70-billion-parameter model for thousands of daily queries requires significant GPU infrastructure investment.

Real-world example: Meta's Llama 2 7B can run on a consumer laptop with 16GB RAM, costing essentially nothing per inference. The same company's Llama 2 70B requires a high-end server GPU setup, costing potentially dollars per query at scale. Both produce high-quality outputs for many tasks, but the deployment economics are radically different.

Practical Applications

Right-sizing AI Models for Specific Applications

A common AI deployment mistake is choosing the largest available model for every task. Matching model size to task complexity and query volume avoids unnecessary cost. Simple classification or extraction tasks often need 1B to 7B parameter models; complex reasoning or generation tasks may benefit from larger models. Benchmark multiple sizes before committing to deployment infrastructure.

Understanding AI API Pricing

Most AI APIs price by token processed — but larger models (more parameters) cost more per token because they require more compute per forward pass. Understanding that price differences between GPT-3.5 and GPT-4 reflect fundamentally different model sizes and capabilities helps organisations make informed decisions about which model to use for which applications.

Evaluating Open-Source vs. Proprietary Model Deployment

Open-source models like Llama 2 and Mistral offer self-hosting options that can be more cost-effective than API-based access for high-volume applications. The parameter count determines the hardware required for self-hosting, enabling informed cost comparisons between self-hosting and API-based deployment.

Best Practices

- Start with the smallest model that meets quality requirements. Benchmark task performance across multiple model sizes before committing to a large model deployment. The smallest model that meets the quality threshold is the most cost-effective choice.
- Understand the memory requirements before planning infrastructure. A rough rule: 2 bytes per parameter in half-precision (FP16) gives minimum GPU memory requirements. Add overhead for context and computation. Use this to plan hardware requirements and cloud infrastructure costs.
- Consider the full cost of ownership, not just API cost. For high-volume applications, self-hosted smaller models may be more economical than API access to larger models. Include infrastructure, maintenance, and scaling costs in the comparison.
- Track parameter count when comparing competing models. When comparing AI products, parameter count is one meaningful proxy for capability — but it must be considered alongside training data quality, fine-tuning approach, and benchmark performance on the specific task.

Common Mistakes and Misconceptions

Misconception: Assuming more parameters always means better performance for every task.

Correct approach: Larger models have more capacity but require more data to realise that capacity. For narrow, well-defined tasks with limited training data, smaller specialised models often outperform much larger general models.

Misconception: Treating parameter count as the primary or only measure of model quality.

Correct approach: Training data quality, fine-tuning approach, alignment techniques, and benchmark performance on the specific task are all equally or more important than raw parameter count for evaluating a model's suitability for a given application.

Misconception: Underestimating the infrastructure implications of large model deployment.

Correct approach: A 70B-parameter model requires roughly 140GB of GPU memory in half-precision — equivalent to two to four enterprise-grade GPUs. This is a significant infrastructure commitment that must be scoped in deployment planning.

Misconception: Believing that emergent abilities in large models means smaller models cannot be effective.

Correct approach: Many business applications do not require emergent cross-domain reasoning abilities. A fine-tuned smaller model is often more than sufficient for well-scoped tasks, and is dramatically more cost-effective to deploy.

Decision Framework: Model Size Selection Guide

Use this guide to inform model size selection for a specific application. Smaller models are preferred unless specific requirements justify a larger one.

Application Type	Recommended Size Range	Rationale
Simple classification or extraction	Under 1B parameters	Task is narrow; large capacity unnecessary and wasteful
Domain-specific Q&A (well-defined)	1B to 7B parameters fine-tuned	Task-specific fine-tuning compensates for smaller size

General-purpose assistant / chatbot	7B to 13B parameters	Good general capability at manageable deployment cost
Complex reasoning or multi-step tasks	13B to 70B parameters	Emergent reasoning abilities begin to appear at this scale
Frontier general AI capabilities	70B+ parameters	Required for tasks needing emergent cross-domain abilities
High-volume, latency-sensitive	Smallest model meeting quality bar	Cost and latency dominate; quality must be benchmarked
Research or capability exploration	Largest available	Exploring frontier capabilities; cost is secondary

How to use this: Always benchmark the smallest model in your size range against your specific task before assuming a larger model is necessary. The performance gap between sizes varies dramatically by task type.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. A simple neural network has 10 input neurons, 5 hidden neurons, and 2 output neurons. How many total parameters does it have?

- A. 17 parameters
- B. 52 parameters
- C. 62 parameters
- D. 72 parameters

Answer: C. 62 parameters

Why: $\text{Parameters} = (\text{input neurons} \times \text{hidden neurons}) + \text{hidden biases} + (\text{hidden neurons} \times \text{output neurons}) + \text{output biases} = (10 \times 5) + 5 + (5 \times 2) + 2 = 50 + 5 + 10 + 2 = 67$. The takeaway: count grows rapidly as layers and neurons increase.

2. A team is building a customer service chatbot to answer questions about a single product line. Which model size is most appropriate?

- A. The largest available model (175B+ parameters) for maximum quality.
- B. A 7B parameter model fine-tuned on product documentation — likely sufficient for well-scoped queries at much lower cost.
- C. A model with no parameters — rule-based systems are always better for narrow tasks.
- D. Parameter count is irrelevant; only the training data matters.

Answer: B. A 7B parameter model fine-tuned on product documentation — likely sufficient for well-scoped queries at much lower cost.

Why: For narrow, well-defined tasks like single-product Q&A, a fine-tuned smaller model is usually sufficient and dramatically more cost-effective than a frontier large model.

3. GPT-2 struggled with multi-step arithmetic, while GPT-3 performs it much better — despite no specific arithmetic training. What phenomenon does this illustrate?

- A. GPT-3 was trained on more arithmetic examples.
- B. Emergent abilities — capabilities that appear above certain parameter count thresholds without being explicitly trained.

- C. GPT-3 uses a fundamentally different architecture.
- D. GPT-3 uses supervised learning while GPT-2 used unsupervised learning.

Answer: B. Emergent abilities — capabilities that appear above certain parameter count thresholds without being explicitly trained.

Why: *Emergent abilities are qualitative capability improvements that appear at scale, not gradual improvements from more examples. Multi-step reasoning is a documented emergent ability in large language models.*

Reflection Questions

1. The trade-off between model size (capability) and deployment cost (resource requirements) mirrors trade-offs in many professional domains. Can you identify analogous trade-offs in your field — where more powerful tools cost proportionally more to deploy?
2. Emergent abilities — capabilities that appear suddenly at scale — challenge our intuition about linear improvement. Can you identify analogies in other fields where quantitative growth produces qualitative change?
3. The fact that smaller fine-tuned models often outperform larger general models on specific tasks has significant implications for AI strategy. What does this suggest about the value of domain-specific data assets versus access to frontier model APIs?
4. GPU memory requirements for large model deployment are a significant infrastructure constraint. How should organisations think about this constraint when making AI adoption decisions?

Key Takeaways

- Model parameters are all the learned weights and biases in a neural network — they collectively encode everything the model has learned from training data.
- Parameter count directly determines model capacity (ability to learn complex patterns), memory requirements (roughly 2 bytes per parameter), inference compute, and training data needs.
- More parameters mean more capacity but also more cost — deployment economics strongly favour the smallest model that meets quality requirements for a given task.
- Emergent abilities appear above certain parameter count thresholds — capabilities that cannot be achieved simply by making smaller models better at specific tasks.
- Smaller fine-tuned models frequently outperform larger general models on narrow, well-defined tasks with sufficient domain-specific training data.
- GPU memory is the primary infrastructure constraint for large model deployment — a 70B-parameter model requires approximately 140GB of GPU memory in half-precision.
- Parameter count is one useful proxy for model capability, but training data quality, fine-tuning, and task-specific benchmarks are equally or more important evaluation criteria.

Recommended Next Actions

5. Calculate the parameter count for a three-layer network of your own design: choose input, hidden, and output sizes, then compute the total using the formula covered in this lecture.
6. Find the parameter count for one AI tool you use and estimate its GPU memory requirement. Is this consistent with whether it runs as an API (cloud) or locally?

7. Add to your glossary: Model Parameters, Model Capacity, Emergent Abilities, Model Distillation, Inference Cost, Fine-tuning.
8. Identify one AI application in your organisation and evaluate: Is the model size appropriate for the task complexity and query volume? What would a right-sized model look like?